

INGENIERIA DE SOFTWARE CON INTELIGENCIA ARTIFICIAL

BACKEND DEVELOPER WEB

TUTOR

MOISES EDUARDO GARCIA FORTUNA

EQUIPO

DIAZ VILLALOBOS JANTZEN

ROLDAN ROCA ANGELO

SILVA ESTRADA FLAVIO

VASQUEZ RODRIGUEZ KORI

2026

Caso:

La empresa TecnoStore S.A.C. vende productos tecnológicos como audífonos, teclados, mouse, cargadores, cámaras web y memorias USB. Actualmente, el área de ventas lleva el control de productos en hojas de cálculo, lo que genera errores frecuentes: productos duplicados, precios desactualizados, dificultades para buscar información y pérdida de tiempo al modificar datos.

La gerencia ha solicitado desarrollar un pequeño módulo web en PHP que permita gestionar productos desde el navegador. El sistema debe permitir buscar productos por descripción, agregar nuevos productos, editar información existente y eliminar registros cuando ya no correspondan.

El equipo de desarrollo propone organizar la solución usando archivos PHP separados para cada operación, una clase Producto encargada de manejar los datos y una conexión a base de datos mediante PDO. Sin embargo, antes de programar, se requiere que los estudiantes analicen la estructura del sistema, identifiquen los elementos técnicos necesarios y propongan una solución ordenada, segura y mantenible.

Actividad 1: Identificación de archivos y responsabilidades

Un sistema de productos en PHP puede estar dividido en los siguientes archivos:

Archivo	Posible responsabilidad
index.php	Es la pagina principal donde vemos los productos
agregar.php	Se añade cualquier producto y se guarda
editar.php	Se modifica los datos de un producto
eliminar.php	Eliminación de un producto
producto.php	Se define si es modelo o clase del producto
conexión.php	Conectamos con la base de datos

Completa la tabla indicando qué función debería cumplir cada archivo dentro del sistema.

Responde

- ¿Por qué es mejor dividir el sistema en varios archivos en lugar de colocar todo en uno solo?

Por que así los desarrolladores pueden organizarse y visualizar las cosas de manera más clara y precisa para evitar atrasos innecesarios y ser mas eficientes con la tarea propuesta.

- ¿Qué archivo debería encargarse de mostrar la lista de productos?

Es el principal “index.php” ya que ahí es la primera pagina que se abre cuando el usuario ingresa y se muestra los productos y la interfaz como los botones o enlaces.

- ¿Qué archivo debería contener la clase Producto?

El archivo "producto.php" para saber diferenciar de diferentes archivos escondidos o erróneos.

- ¿Qué archivo debería encargarse únicamente de la conexión con MySQL?

El archivo "conexión.php" ya que ahí se ingresan los datos, y este mismo se encarga de crear y administrar los datos.

- ¿Qué riesgos existen si se mezcla la conexión, la lógica de negocio y el diseño HTML en un solo archivo?

Tendríamos varios errores al copilar junto con el mantenimiento y no se podría trabajar en equipo.

Actividad 2: Organización por capas

Relaciona cada elemento con la capa que corresponde:

Elemento	Capa sugerida: Vista / Modelo / Configuración
Formulario para buscar productos	Vista
Clase Producto	Modelo
Archivo de conexión a MySQL	Configuración
Método setInsertar()	Modelo
Método getBuscarPorDescripcion()	Modelo
Tabla HTML de resultados	Vista

Preguntas de análisis

- ¿Qué ventaja tiene separar la vista del modelo?

Permite que los archivos no estén mezclados entre sí y puedan ser modificados de manera independiente sin alterar algunos de procesos de la base de datos.

- ¿Por qué la conexión a la base de datos no debería repetirse en cada archivo?

Esto podría hacer que halla fallas en el sistema sea porque se dupliquen o por seguridad, ocasionando sobrecarga.

- ¿Cómo ayuda esta organización al mantenimiento del sistema?

Facilita encontrar errores, realizar cambios, agregar nuevas funcionalidades y reutilizar código sin afectar otras partes del sistema.

Preguntas de pensamiento crítico

- Si el sistema funciona, aunque todo esté en un solo archivo, ¿sería correcto dejarlo así? Justifica.

No es recomendable. Aunque funcione inicialmente, cuando el sistema crezca será más difícil mantenerlo, corregir errores y agregar nuevas funcionalidades. Una estructura organizada mejora la escalabilidad y la calidad del software.

- ¿Qué problemas aparecerían si en el futuro se agregan clientes, ventas y categorías?

El archivo único crecería demasiado, habría código repetido, aumentaría la dificultad para localizar errores y sería más complicado trabajar en equipo o implementar nuevas funciones.

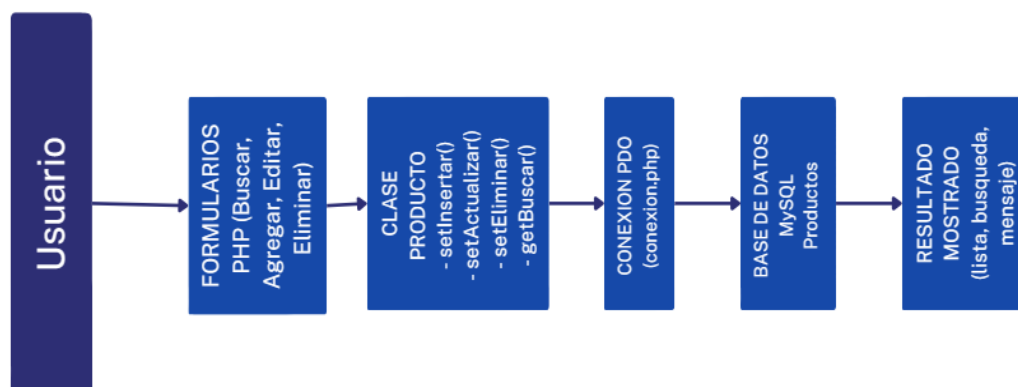
- ¿Qué decisión técnica permitiría que otro programador entienda el sistema con mayor facilidad?

Organizar el sistema por capas (Vista, Modelo y Configuración), utilizar nombres descriptivos para archivos y métodos, y mantener cada archivo con una responsabilidad específica.

Esquema solicitado

Elabora un esquema por capas del sistema de productos. Debe incluir:

- Usuario.
- Formularios PHP.
- Clase Producto.
- Conexión PDO.
- Base de datos MySQL.
- Resultado mostrado en pantalla



Actividad 3: Reconocimiento de controles de formulario

Un formulario de producto tiene los siguientes campos:

Campo del formulario	Nombre del control	Dato que almacena
Código del Producto	txtId	Código o identificador del producto
Descripción	txtDescripcion	Texto o cadena de caracteres
Categoría	txtCategoria	Texto o id que hace referencia a la categoría
Precio	txtPrecio	Precio del producto
Texto de búsqueda	txtDescripcionBuscar	Descripcion o texto para buscar el producto

Completa la tabla indicando que dato representa cada color.

Responde

- ¿Para que sirve el atributo name en un formulario HTML?

Sirve para identificar el dato en el servidor PHP mediante los arreglos \$_POST o \$_GET, sin este atributo, el valor no se envía.

- ¿Por qué el campo código podría estar marcado como readonly?

Para evitar modificaciones accidentales del ID único durante una edición, garantizando la integridad de los datos.

- ¿Qué diferencia hay entre un campo de texto para buscar y un campo de texto para registrar?

El de buscar es opcional, acepta datos parciales y usa GET. El de registrar es obligatorio, exige datos exactos y usa POST.

- ¿Qué validaciones deberían aplicarse al precio?

Debe ser numérico, mayor que cero, aceptar máximo dos decimales y ser un campo obligatorio.

Actividad 4: Envío de datos mediante GET

Analiza la siguiente situación:

Un formulario envía los datos del producto a través de la URL usando el método GET. Luego PHP recibe esos datos mediante \$_GET.

Completa la tabla:

Dato recibido	Variable PHP sugerida	Validación necesaria
txtId	\$id	Ver que exista y que sea numérico
txtDescripcion	\$descripcion	Que no este vacío y no tenga caracteres peligrosos
txtCategoria	\$categoria	Que no esté vacío

txtPrecio	\$precio	Que sea numérico y mayor a cero
txtDescripcionBuscar	\$buscar	Ver que exista y limpiar

Responde:

- ¿Qué es \$_GET en PHP?

Es un arreglo global de PHP que permite acceder a los datos enviados desde un formulario o una URL mediante el método GET.

- ¿Qué significa verificar isset(\$_GET["txtId"])?

Consiste en validar que el parámetro txtId haya sido recibido correctamente antes de intentar utilizarlo en el programa.

- ¿Qué ocurre si se intenta leer un dato que no fue enviado?

PHP mostrará una advertencia indicando que la clave no existe, lo que puede provocar comportamientos inesperados en la aplicación.

- ¿Qué diferencia existe entre usar GET y POST?

Con GET la información se envía como parte de la dirección web y puede ser vista por el usuario, mientras que con POST los datos se transmiten de forma interna en la solicitud, ofreciendo mayor privacidad y capacidad de envío.

- ¿Qué método sería más recomendable para registrar o actualizar productos? Justifica.

Se recomienda utilizar POST, ya que protege mejor la información al no mostrarla en la URL, admite una mayor cantidad de datos y reduce el riesgo de envíos involuntarios al actualizar la página.

Preguntas de análisis

- ¿Por qué es importante verificar si una variable existe antes de usarla?

Porque permite asegurar que los datos requeridos están disponibles y evita errores que puedan afectar el funcionamiento del sistema.

- ¿Qué errores podrían aparecer si el usuario deja campos vacíos?

Podrían generarse problemas al guardar la información en la base de datos o resultados incorrectos en operaciones que dependan de esos datos.

- ¿Por qué no se debe confiar completamente en los datos enviados desde un formulario?

Porque los datos pueden ser modificados por usuarios malintencionados, quienes podrían ingresar información incorrecta o intentar realizar ataques contra la aplicación.

Preguntas de pensamiento crítico

- Si el precio se envía por la URL, ¿qué riesgo podría existir?

El principal riesgo es que cualquier persona podría ver y cambiar el precio directamente desde la URL antes de enviarlo. Esto podría provocar que se registren valores incorrectos o manipulados en el sistema.

- ¿Sería correcto permitir que el usuario modifique el código del producto desde el formulario? Justifica.

No sería una buena idea, porque el código identifica de manera única a cada producto. Si un usuario lo cambia, podrían aparecer productos duplicados o problemas para encontrar y actualizar la información correcta.

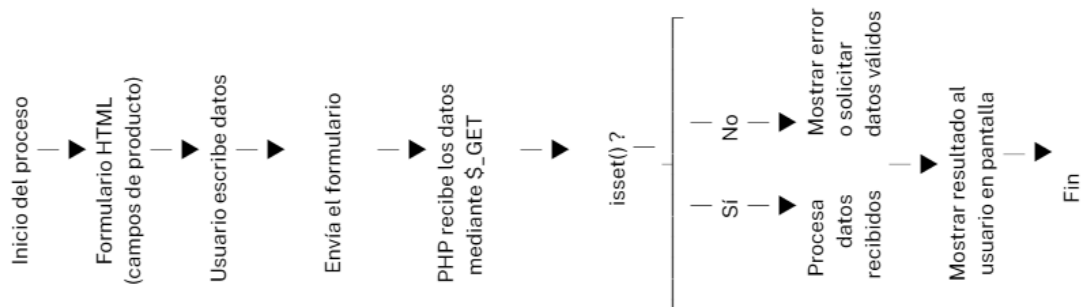
- ¿Qué impacto tendría registrar productos sin validar descripción, categoría y precio?

El sistema podría guardar información incorrecta o incompleta, como productos sin nombre, sin categoría o con precios inválidos. Esto dificultaría la gestión de los productos y podría generar errores en las ventas y reportes.

Esquema solicitado

Elabora un diagrama de flujo del envío de datos:

Formulario HTML → Usuario escribe datos → Envía formulario → PHP recibe con \$_GET → Verifica con isset() → Procesa información.



Actividad 5: Análisis de la clase Producto

Una clase Producto puede tener las siguientes propiedades:

Propiedad	Descripción	Tipo de dato sugerido
id	Identificador único del producto	int
descripcion	Nombre comercial del producto	string
categoria	Grupo al que pertenece el producto	string

precio	Costo de venta al público del producto	Float o double
--------	--	----------------

Completa la tabla.

Responde:

- ¿Qué representa una clase en PHP?

Una clase es una estructura que sirve como base para crear objetos. En ella se definen las características y acciones que tendrán esos objetos.

- ¿Qué representa un objeto?

Un objeto es un elemento creado a partir de una clase. Por ejemplo, si la clase es Producto, un objeto podría ser un teclado, un mouse o una memoria USB con sus propios datos.

- ¿Por qué tendría sentido crear una clase llamada Producto?

Porque permite reunir en un solo lugar toda la información y las operaciones relacionadas con los productos, haciendo que el código sea más organizado y fácil de mantener.

- ¿Qué relación existe entre la tabla producto de la base de datos y la clase Producto en PHP?

La tabla almacena la información de los productos en la base de datos, mientras que la clase permite trabajar con esos datos dentro del programa. Ambas representan la misma información, pero en entornos diferentes.

Actividad 6: Métodos de la clase

La clase Producto contiene métodos relacionados con el CRUD:

Método	Operación que realiza	Tipo de operación CRUD
getBuscarPorId(\$idBuscar)	Buscar un producto mediante su código	Read (leer)
getBuscarPorDescripcion(\$descripcionBuscar)	Buscar productos por su descripción	Read(leer)
setInsertar(\$producto)	Registrar un nuevo producto en la base de datos	Create (crear)
setActualizar(\$producto)	Modifica los datos de un	Update(actualizar)

	producto existente	
setEliminar(\$idEliminar)	Elimina un producto de la base de datos	Delete(eliminar)

Responde:

- ¿Por qué algunos métodos empiezan con get?

Porque se utilizan para obtener, consultar o recuperar información de la base de datos sin modificarla.

- ¿Por qué algunos métodos empiezan con set?

Porque se utilizan para modificar datos, ya sea insertando, actualizando o eliminando registros.

- ¿Qué método se usaría para mostrar los datos antes de editarlos?

Se utilizaría el método `getBuscarPorId($idBuscar)`, ya que permite obtener los datos del producto que será modificado

- ¿Qué método se usaría para guardar un producto nuevo?

Se utilizaría el método `setInsertar($producto)`, porque registra un nuevo producto en la base de datos.

- ¿Qué método se usaría para eliminar un producto?

Se utilizaría el método `setEliminar($idEliminar)`, porque elimina el producto seleccionado.

Preguntas de análisis

- ¿Por qué es conveniente que la clase Producto concentre las operaciones relacionadas con productos?

Porque permite mantener el código organizado, reutilizable y fácil de mantener. Todas las funciones relacionadas con productos se encuentran en un solo lugar.

- ¿Qué pasaría si la búsqueda, inserción, actualización y eliminación estuvieran dispersas en varios archivos?

El sistema sería más difícil de entender, mantener y corregir. Además, aumentaría la posibilidad de errores y duplicación de código.

- ¿Qué ventaja ofrece trabajar con objetos en lugar de solo variables sueltas?

Los objetos permiten agrupar datos y comportamientos en una sola estructura, mejorando la organización, reutilización y mantenimiento del sistema.

Preguntas de pensamiento crítico

- ¿Es recomendable declarar las propiedades como públicas?

No es recomendable, porque cualquier parte del programa podría modificar los datos directamente. Lo más adecuado es declararlas como privadas (private) y acceder a ellas mediante métodos getter y setter.

- ¿Qué alternativa sería más adecuada desde la programación orientada a objetos?

La alternativa más adecuada es implementar el patrón MVC (Modelo-Vista-Controlador). Esto consiste en crear una clase **Producto** que use encapsulamiento para manejar la lógica del negocio, una clase **Conexion** que centralice el acceso seguro mediante PDO, y separar por completo el diseño HTML de las consultas a la base de datos.

- ¿Por qué una clase debería representar tanto datos como comportamiento?

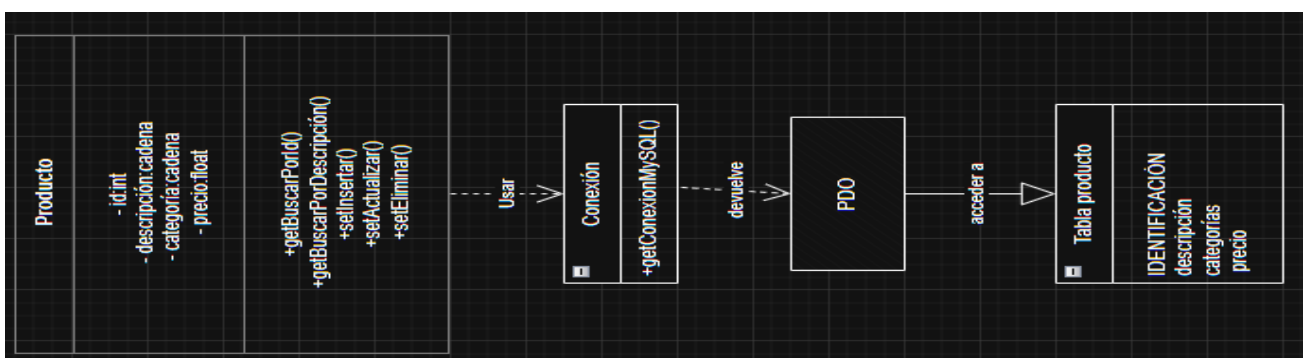
Porque una clase no solo almacena información, sino que también define las acciones que pueden realizarse sobre esos datos, siguiendo los principios de la programación orientada a objetos.

- ¿Qué mejoras aplicarías a la clase Producto para que sea más segura y ordenada?
 - Utilizar propiedades privadas.
 - Validar los datos antes de guardarlos.
 - Implementar manejo de excepciones.
 - Utilizar consultas preparadas con PDO.
 - Agregar comentarios y documentación al código.
 - Separar la lógica de negocio de la interfaz de usuario.

Esquema solicitado

Elabora un diagrama de clase simple para Producto, incluyendo:

- Propiedades.
- Métodos.
- Relación con la tabla producto.
- Relación con la conexión PDO.



Actividad 7: Interpretación de la conexión

Un sistema PHP puede obtener la conexión a MySQL mediante un método como:

\$cnx = Conexion::getConexionMySQL();

Responde:

- ¿Qué representa \$cnx?

Representa el objeto de conexión a la base de datos MySQL. A través de esta variable se pueden ejecutar consultas y operaciones sobre la base de datos utilizando PDO.

- ¿Qué significa llamar a un método usando ::?

El operador :: permite acceder a métodos o propiedades estáticas de una clase sin necesidad de crear un objeto de dicha clase.

- ¿Por qué el método de conexión podría ser estático?

Porque la conexión puede utilizarse directamente desde cualquier parte del sistema sin necesidad de instanciar la clase Conexion, facilitando su reutilización.

- ¿Qué ventaja tiene centralizar la conexión en una clase Conexion?

Permite reutilizar el mismo código de conexión, evita duplicar información en varios archivos y facilita el mantenimiento y las modificaciones futuras.

- ¿Qué datos básicos se necesitan para conectar PHP con MySQL?

Se necesita el servidor, nombre de la base de datos, usuario, contraseña y el juego de caracteres utilizado para la conexión.

Completa:

Dato necesario	Ejemplo
Servidor	127.0.0.1
Base de datos	bdmarket
Usuario	root
Contraseña	root
Juego de caracteres	Utf8

Actividad 8: PDO y manejo de errores

Responde:

- ¿Qué es PDO?

PDO (PHP Data Objects) es una extensión de PHP que permite conectarse y trabajar con bases de datos de manera segura y eficiente mediante objetos.

- ¿Qué ventaja tiene PDO frente a escribir consultas de forma directa e insegura?

PDO permite utilizar consultas preparadas y parámetros, reduciendo el riesgo de ataques por inyección SQL y facilitando el mantenimiento del código.

- ¿Por qué es recomendable usar excepciones para manejar errores?

Porque permiten detectar, controlar y mostrar mensajes adecuados cuando ocurre un error, evitando que el sistema falle de forma inesperada.

- ¿Qué tipo de errores podrían ocurrir durante la conexión?
 - Servidor MySQL apagado o inaccesible.
 - Base de datos inexistente.
 - Usuario incorrecto.
 - Contraseña incorrecta.
 - Problemas de configuración o codificación de caracteres.
- ¿Qué mensaje debería ver el usuario si la conexión falla?

Se debería mostrar un mensaje sencillo como:

"Error al conectar con la base de datos. Intente nuevamente más tarde."

Completa la tabla:

Error posible	Causa probable	Acción recomendada
No conecta con MySQL	El servidor MySQL está apagado o el puerto es incorrecto	Verificar que el servicio MySQL esté iniciado y revisar la configuración
Base de datos inexistentes	El nombre de la base de datos es incorrecto o no existe	Crear la base de datos o corregir el nombre
Usuario incorrecto	El usuario configurado no existe	Verificar las credenciales de acceso
Contraseña incorrecta	La contraseña ingresada es errónea	Corregir la contraseña configurada
Error de codificación de caracteres	Configuración incorrecta del charset	Configurar correctamente UTF-8 u otra codificación adecuada

Preguntas de análisis

- ¿Qué diferencia existe entre un error de conexión y un error de consulta?

Un error de conexión ocurre cuando no se puede establecer comunicación con la base de datos. Un error de consulta ocurre cuando la conexión existe, pero la instrucción SQL contiene errores o no puede ejecutarse correctamente.

- ¿Por qué no se debe mostrar al usuario final toda la información técnica del error?

Porque podría exponer información sensible del sistema, como nombres de bases de datos, usuarios, contraseñas o rutas internas, generando riesgos de seguridad.

- ¿Qué ventaja tiene reutilizar una misma conexión en diferentes operaciones?

Reduce la duplicación de código, mejora el rendimiento y facilita el mantenimiento del sistema.

Preguntas de pensamiento crítico

- Si una conexión falla, ¿el sistema debería detenerse o continuar? Justifica según el caso.

Si la operación depende de la base de datos, el sistema debe detener esa operación y mostrar un mensaje de error controlado. Continuar sin conexión podría generar resultados incorrectos o pérdida de información.

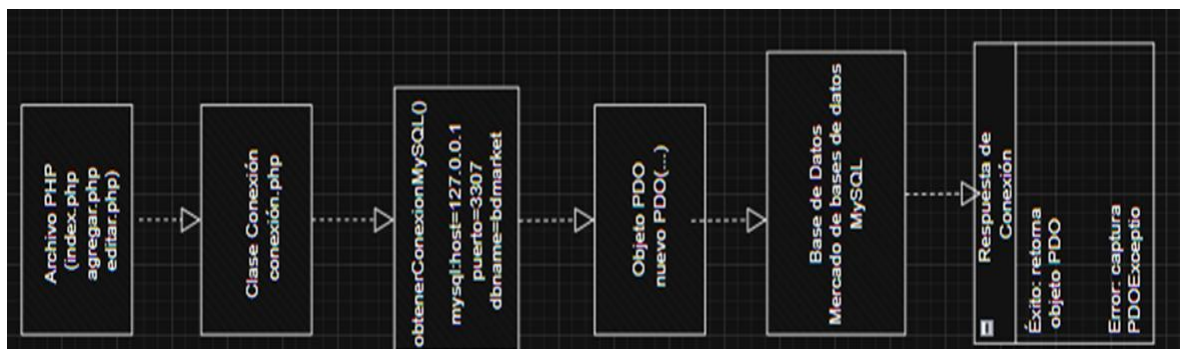
- ¿Qué información técnica podría ser peligrosa si se muestra en pantalla?
 - Nombre del servidor.
 - Nombre de la base de datos.
 - Usuario de conexión.
 - Contraseñas.
 - Consultas SQL completas.
- ¿Qué impacto tendría un sistema de ventas que no controle adecuadamente sus errores de base de datos?

Podría provocar pérdida de información, registros incorrectos, interrupción del servicio, problemas en las ventas y una mala experiencia para los usuarios, afectando la confiabilidad del negocio.

Esquema solicitado

Elabora un mapa de proceso de conexión PDO:

- Archivo PHP → Clase Conexion → Método de conexión → Objeto PDO → Base de datos MySQL → Respuesta de conexión.



Actividad 9: Análisis de búsqueda por código

Observa la siguiente estructura lógica:

- `$snt = $cnx->prepare("SELECT * FROM producto WHERE id = :idBuscar");`
- `$snt->bindValue(":idBuscar", $idBuscar);`
- `$snt->execute();`
- `$fila = $snt->fetch();`

Responde:

- ¿Qué hace `prepare()`?

Prepara una consulta SQL antes de ejecutarla, permitiendo el uso de parámetros para mejorar la seguridad.

- ¿Qué representa: `idBuscar`?

Es un parámetro o marcador de posición que recibirá el valor del ID que se desea buscar.

- ¿Qué hace `bindValue()`?

Asigna un valor a un parámetro de la consulta preparada.

- ¿Qué hace `execute()`?

Ejecuta la consulta SQL preparada en la base de datos.

- ¿Qué hace `fetch()`?

Obtiene una fila o registro del resultado de la consulta.

- ¿Qué diferencia hay entre preparar una consulta y ejecutarla?

Preparar una consulta significa definir la instrucción SQL y sus parámetros. Ejecutarla significa enviarla a la base de datos para obtener resultados

Completa:

Elemento	Función
<code>\$snt</code>	Objeto que almacena la consulta preparada
<code>prepare()</code>	Prepara la consulta SQL
<code>:idBuscar</code>	Parámetro que recibirá el valor del ID
<code>bindValue()</code>	Asigna un valor al parámetro
<code>execute()</code>	Ejecuta la consulta
<code>fetch()</code>	Obtiene un registro del resultado

Actividad 10: Búsqueda por descripción

Una búsqueda por descripción puede usar una consulta similar:

- `SELECT * FROM producto`
- `WHERE descripcion LIKE CONCAT('%', :descripcionBuscar, '%');`

Responde:

- ¿Para qué sirve LIKE?

Sirve para buscar registros que coincidan total o parcialmente con un texto dentro de una consulta SQL.

- ¿Para qué se usa % en una búsqueda?

El símbolo % es un comodín que representa cualquier cantidad de caracteres antes o después del texto buscado.

- ¿Qué significa buscar coincidencias parciales?

Significa encontrar registros que contengan una parte del texto ingresado, sin necesidad de que coincida exactamente.

- ¿Por qué conviene usar parámetros incluso en búsquedas?

Porque aumentan la seguridad del sistema, evitando ataques de inyección SQL y facilitando el manejo de datos enviados por el usuario.

- ¿Qué resultado se espera si el usuario escribe solo una parte del nombre del producto?

Se mostrarán todos los productos cuya descripción contenga ese texto.

Completa la tabla:

Texto ingresado por el usuario	Resultado esperado
mouse	Muestra productos que contengan la palabra "mouse"
tecl	Muestra productos como "teclado" o similares
usb	Muestra productos relacionados con USB
Texto vacío	Muestra todos los productos o ningún filtro aplicado
Texto inexistente	No devuelve resultados

Preguntas de análisis

- ¿Cuál es la diferencia entre fetch() y fetchAll()?
 - fetch() obtiene un solo registro de la consulta.
 - fetchAll() obtiene todos los registros encontrados.
- ¿Por qué una búsqueda por código devuelve normalmente un solo registro?

Porque el código o ID es único para cada producto, por lo que solo existe un resultado.

- ¿Por qué una búsqueda por descripción puede devolver varios registros?

Porque varios productos pueden contener palabras similares o coincidir con el texto buscado.

- ¿Qué ocurre si no se encuentra ningún producto?

La consulta no devuelve registros y el sistema debe informar que no existen resultados para la búsqueda realizada.

Preguntas de pensamiento crítico

- ¿Qué riesgo existe si se arma la consulta SQL concatenando directamente lo que escribe el usuario?

Existe el riesgo de sufrir ataques de inyección SQL, donde un usuario malintencionado podría ejecutar instrucciones no autorizadas sobre la base de datos.

- ¿Por qué las consultas preparadas ayudan a prevenir inyección SQL?

Porque separan la consulta SQL de los datos ingresados por el usuario, evitando que estos sean interpretados como código SQL.

- ¿Es suficiente usar bindValue() o también se deben validar los datos? Justifica.

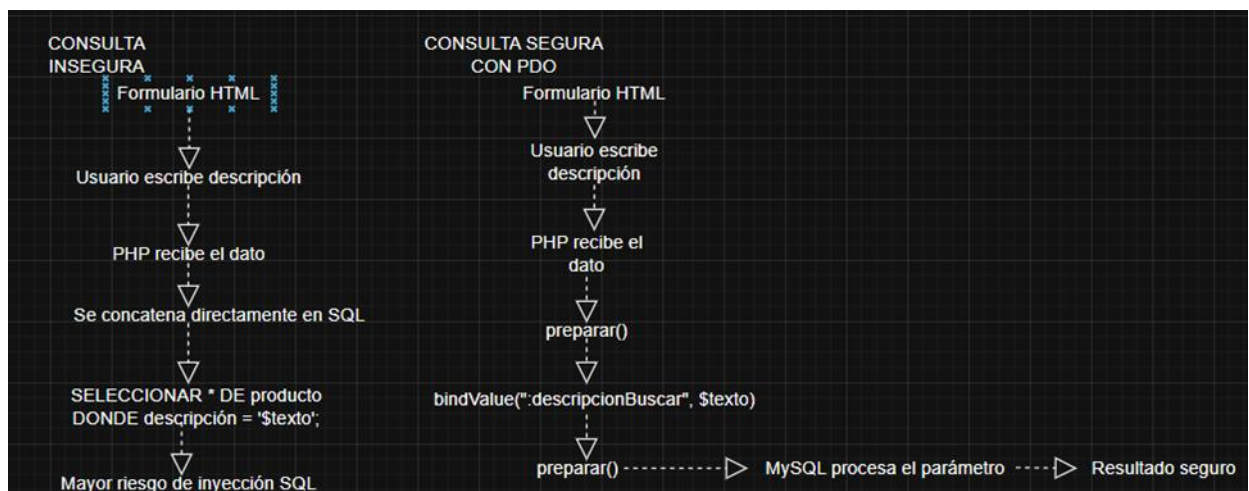
No. Además de usar bindValue(), también se deben validar los datos para verificar que tengan el formato correcto, evitar valores vacíos y garantizar la integridad de la información almacenada.

Esquema solicitado

Elabora un esquema comparativo:

Consulta insegura	Consulta segura con PDO
Datos se conectan directamente	Datos se envían como parámetros
Mayor riesgo de inyección SQL	Menor riesgo de inyección SQL
Difícil de mantener	Más ordenada y controlada

Agrega flechas o pasos que muestren cómo viajan los datos desde el formulario hasta la consulta.



Actividad 11: Relacionar acciones con sentencias SQL

Completa la tabla:

Acción del usuario	Operación CRUD	Sentencia SQL principal	Método sugerido en la clase
Buscar productos por descripción	READ	select * from producto where descripcion like concat('%', :desc, '%')	getConexionMySQL(), prepare(), bindValue(), execute(), fetchAll(), array_push()
Buscar productos por id	READ	select * from producto where id=:idSearch	getConexionMySQL(), prepare(), bindValue(), execute(), fetch()
Agregar producto	CREATE	insert into producto (descripcion, categoria, precio) values (:descripcion, :categoria, :precio)	getConexionMySQL(), prepare(), bindValue(), execute(), lastInsertId()
Editar producto	UPDATE	update producto set descripcion = :descripcion, categoria = :categoria, precio = :precio where id = :id	getConexionMySQL(), prepare(), bindValue(), execute()
Eliminar producto	DELETE	delete from producto where id = :id	getConexionMySQL(), prepare(), bindValue(), execute()

Actividad 12: Análisis de cada operación

Responde para cada operación:

Operación	¿Qué datos necesita?	¿Qué validaciones requiere?	¿Qué podría fallar?
Buscar por descripción	string \$descSearch	Limpieza de espacios en blanco, evitar	Que no existan coincidencias o la base de datos esté fuera de

		caracteres vacíos.	servicio.
Buscar por id	int \$idSearch	Verificar que sea un número entero positivo.	Que el ID solicitado no exista en la tabla.
Insertar producto	Producto \$producto	Campos obligatorios no vacíos, tipos de datos (numéricos positivos).	Duplicidad en campos únicos o violación de restricciones de tipo.
Actualizar producto	Producto \$producto	Validar que el ID exista, datos nuevos válidos con formato correcto.	Que el ID haya sido borrado simultáneamente por otro usuario.
Eliminar producto	int \$id	Confirmar existencia del ID, verificar restricciones de integridad.	Error de integridad referencial (que esté en uso en otra tabla).

Preguntas de análisis

- ¿Por qué insertar un producto no debería incluir manualmente el código si este es autogenerado?

Porque la base de datos gestiona el identificador automáticamente mediante la propiedad AUTO_INCREMENT. Si se intenta ingresar manualmente, se corre el riesgo de generar colisiones de datos (duplicidad de ID) o romper la secuencia lógica del sistema.

- ¿Por qué después de insertar puede ser necesario recuperar el nuevo ID?

Es sumamente necesario para la experiencia de usuario y operaciones encadenadas; por ejemplo, para redireccionar inmediatamente a la vista de detalle del producto recién creado o para insertar registros dependientes en tablas relacionadas.

- ¿Qué diferencia existe entre actualizar todos los datos del producto y actualizar solo el precio?

Actualizar todos los datos requiere procesar y validar un formulario completo, lo que genera mayor tráfico e impacto en la fila. Modificar solo el precio (operación atómica) es un proceso más específico, rápido y con menor riesgo de alterar accidentalmente datos estáticos como la descripción.

- ¿Qué precauciones deben tomarse antes de eliminar un producto?

Se debe comprobar la integridad referencial (que el producto no esté amarrado a facturas o ventas) y pedir confirmación explícita al usuario para evitar pérdidas accidentales.

Preguntas de pensamiento crítico

- ¿Sería mejor eliminar físicamente un producto o marcarlo como inactivo? Justifica.

Es mejor marcarlo como inactivo (Eliminado lógico). La eliminación física (DELETE) destruye el historial métrico y corrompe la integridad de datos históricos en reportes de ventas anteriores. El borrado lógico (UPDATE productos SET estado = 0) preserva la consistencia de los datos.

- ¿Qué pasaría si se elimina un producto que ya fue usado en una venta?

Si la base de datos tiene llaves foráneas configuradas correctamente, el motor bloqueará la eliminación arrojando un error de Restricción de Integridad Referencial. Si no tiene estas restricciones, se generarán registros huérfanos en la tabla de ventas, arruinando los reportes financieros.

- ¿Qué operación CRUD consideras más crítica para controlar y por qué?

Delete (Eliminar). Es la operación más crítica porque sus efectos suelen ser destructivos e irreversibles (si es físico). Un error en su control puede resultar en la pérdida definitiva de activos de información empresarial.

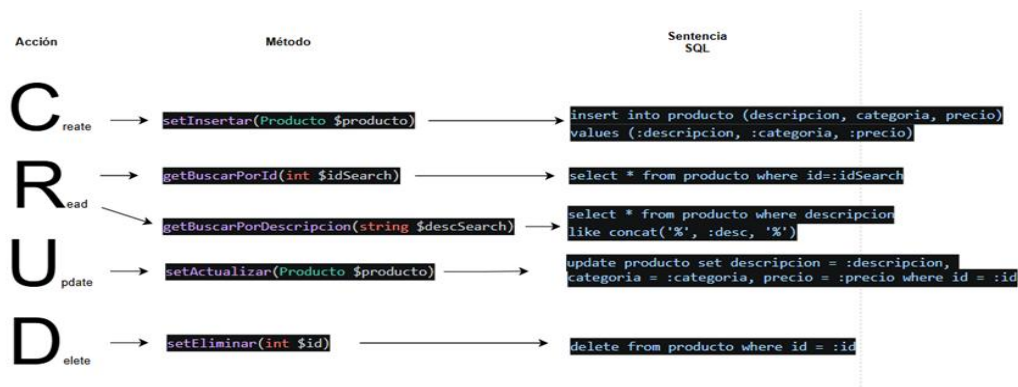
- ¿Qué validaciones mínimas debería tener el sistema antes de insertar o actualizar?

Control de campos requeridos (no nulos), verificación de tipos de datos adecuados (números para stock/precio, cadenas de longitud adecuada para nombres) y sanitización contra scripts maliciosos (XSS).

Esquema solicitado

Elabora un diagrama CRUD del módulo producto:

Crear: agregar producto Leer: buscar por ID y buscar por descripción. Actualizar: editar producto. Eliminar: eliminar producto.



Actividad 13: Análisis de navegación

Un sistema básico de productos puede tener enlaces como:

Enlace o botón	Acción esperada
Agregar	Redirecciona a un formulario limpio (insertar.php) para crear un registro.
Editar	¿Envía el ID por la URL hacia un formulario cargado con los datos actuales (actualizar.php?id=...).
Eliminar	Dispara un evento de confirmación y procesa la baja del registro.
Guardar	Envía los datos del formulario al servidor para procesar la persistencia en la base de datos.
Retornar	Cancela la acción actual y regresa al listado general de registros (index.php).
Buscar	Envía la palabra clave para filtrar la tabla y renderizar los resultados que coincidan.

Luego responde:

- ¿Qué diferencia hay entre un enlace y un botón de formulario?

Un enlace (<a>) sirve principalmente para la navegación directa entre páginas (peticiones GET), mientras que un botón (<button type="submit">) se encarga de empaquetar y despachar los datos cargados en los campos de un formulario hacia el servidor (habitualmente a través de peticiones POST).

- ¿Por qué el enlace Editar debe enviar el ID del producto?

Porque el servidor necesita conocer de manera inequívoca qué registro específico se pretende modificar para realizar la consulta respectiva y rellenar los campos del formulario con los valores correctos.

- ¿Por qué el enlace Eliminar debe confirmar la acción antes de ejecutarse?

Para actuar como una barrera de seguridad contra clicks o pulsaciones accidentales del usuario, evitando eliminaciones involuntarias y destructivas.

- ¿Qué debería ocurrir después de guardar un producto?

El servidor debe procesar el guardado y retornar el id del nuevo registro, mostrando al usuario el registro en la página.

- ¿Qué debería ocurrir después de eliminar un producto?

Debe redireccionar de manera inmediata al listado general actualizando la vista, reflejando que el registro ya no figura en el sistema.

Actividad 14: Confirmación con JavaScript

Analiza esta situación:

Antes de eliminar un producto, el sistema muestra un mensaje:

“¿Está seguro que desea eliminar?”

Responde:

- ¿Por qué es importante confirmar una eliminación?

Es vital porque previene la pérdida accidental de datos operativos de la empresa.

- ¿Qué podría pasar si el usuario elimina por error?

Si el usuario borra por error, podría detener operaciones comerciales, perder el historial de un cliente o requerir largas tareas de restauración desde copias de respaldo.

- ¿Qué función cumple JavaScript en esta parte del sistema?

Funciona como la primera línea de validación del lado del cliente (Client-Side). Intercepta el click del usuario mediante ventanas de confirmación (confirm()) y detiene el envío de la petición al servidor si la respuesta es negativa, ahorrando recursos de red.

- ¿La confirmación en JavaScript reemplaza las validaciones del servidor? Justifica.

En absoluto. JavaScript puede ser desactivado fácilmente en el navegador por cualquier usuario o saltado mediante herramientas como Postman. La seguridad y validación definitiva e ineludible siempre debe residir en el servidor (PHP).

- ¿Qué validaciones debería hacer PHP aunque JavaScript confirme la acción?

Verificar que el ID recibido por parámetro sea un número válido, comprobar que el registro realmente exista en la base de datos y chequear restricciones de integridad referencial.

Preguntas de análisis

- ¿Por qué una operación de eliminación requiere más cuidado que una búsqueda?

La búsqueda es una operación de lectura pasiva que no altera el estado del sistema. La eliminación es una acción destructiva de escritura que modifica la base de datos permanentemente y puede desencadenar fallos de integridad si no se controla.

- ¿Qué información debería mostrarse al usuario antes de confirmar?

Datos clave que permitan identificar plenamente qué se está borrando, tales como el código único, el nombre exacto del producto y una advertencia clara del impacto de la acción.

- ¿Por qué el sistema debe redireccionar luego de eliminar?

Para limpiar los parámetros de la URL (?id=...) y forzar la recarga del listado limpio, previniendo que una recarga accidental del navegador intente ejecutar de nuevo la acción de borrado sobre un ID inexistente.

Preguntas de pensamiento crítico

- ¿Es suficiente preguntar “¿Está seguro?” antes de eliminar? ¿Qué otra información debería mostrarse?

No basta en sistemas complejos. Debería mostrarse qué elementos asociados se verán afectados o, idealmente en flujos de alta importancia, exigir que el usuario escriba la palabra "ELIMINAR" o la clave del producto para ratificar la acción.

- ¿Qué pasaría si un usuario manipula la URL y envía manualmente un ID para eliminar?

Si el sistema es básico y carece de validaciones de sesión o tokens de seguridad, procederá a eliminar el registro solicitado de manera indiscriminada. Esto es una vulnerabilidad grave llamada IDOR (Insecure Direct Object Reference).

- ¿Qué controles de seguridad faltan en un sistema básico antes de permitir eliminar registros?

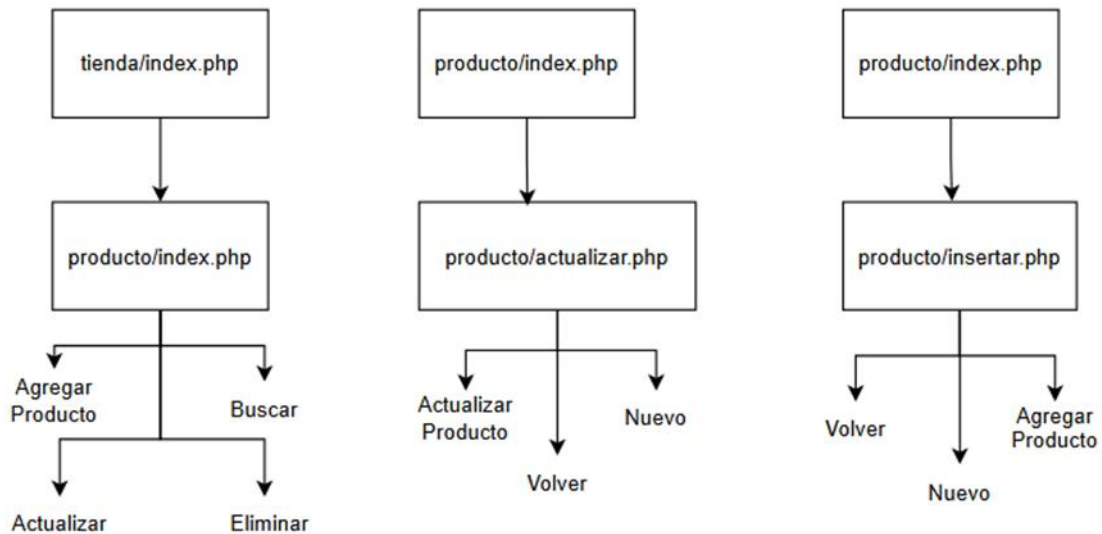
Falta control de autenticación (saber quién es el usuario), autorización basada en roles (solo administradores deberían borrar), y la implementación de Tokens CSRF para certificar que la petición de borrado se originó legítimamente desde los formularios del propio sistema.

Esquema solicitado

Elabora un diagrama de navegación entre páginas:

- index.php → agregar.php → guardar → retornar
- index.php → editar.php?id=... → actualizar → retornar
- index.php → confirmar eliminación → eliminar.php?id=... → redireccionar a index.php

diagrama de navegación entre páginas



4. Caso de pensamiento crítico

Lee la siguiente afirmación:

“Como el sistema es pequeño, no importa si los datos se envían por GET, si las propiedades son públicas y si se elimina directamente el producto de la base de datos. Lo importante es que el sistema funcione.”

Responde:

- ¿Qué errores técnicos presenta esta afirmación?

Uso indiscriminado de GET: Las peticiones GET quedan registradas en el historial del navegador, servidores proxy y logs del servidor, lo que expone los datos. Además, los motores de búsqueda pueden indexar estos enlaces y causar borrados masivos involuntarios.

Propiedades públicas sin control: Rompe el principio de encapsulamiento de la POO, permitiendo alterar el estado interno de un objeto saltándose las capas de validación.

Eliminación directa (Física): Pone en grave riesgo la integridad de la base de datos por los motivos expuestos anteriormente.

- ¿Por qué un sistema pequeño también debe desarrollarse con buenas prácticas?

Porque los sistemas pequeños suelen crecer con el tiempo. El código mal diseñado se vuelve insostenible (deuda técnica), propenso a fallos catastróficos de seguridad y sumamente costoso de refactorizar o escalar en el futuro.

- ¿Qué riesgo existe al enviar datos sensibles por GET?

Exposición de datos e interceptación. La información viaja visible en la barra de direcciones (URL), quedando expuesta a ataques de hombro (shoulder surfing), registros en cachés de red o ataques de fuga de datos a través del campo Referer de HTTP.

- ¿Qué problema puede generar el uso de propiedades públicas sin validación?

Permite que cualquier componente externo de la aplicación asigne valores ilógicos, corruptos o maliciosos a los atributos del objeto (ej. poner un precio negativo o un nombre vacío) evadiendo las reglas de negocio establecidas en los métodos setters.

- ¿Por qué eliminar directamente un producto puede afectar la integridad de la información?

Provoca la ruptura de la coherencia histórica de los datos comerciales. Desaparecen los registros vinculados en cascada o se rompen relaciones clave, invalidando balances financieros y auditorías de inventario del pasado.

- ¿Qué mejoras propondrías para hacer el sistema más seguro y mantenible?

Cambiar los flujos de alteración de datos (Insert, Update, Delete) estrictamente a métodos POST.

Implementar encapsulamiento riguroso en clases PHP usando propiedades private o protected administradas mediante métodos getters y setters con validación integrada.

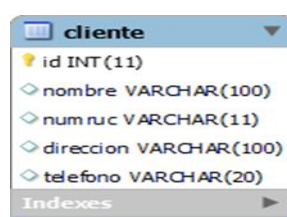
Migrar el borrado físico hacia un sistema de borrado lógico (soft-delete) basado en estados/estatus.

- ¿Qué diferencia existe entre un sistema que "funciona" y un sistema correctamente diseñado?

Un sistema que solo "funciona" cumple con la tarea en un escenario ideal controlado, pero es frágil, vulnerable a exploits sencillos y colapsa ante imprevistos. Un sistema correctamente diseñado es seguro, escalable, robusto ante fallos, fácil de probar y mantenible por otros desarrolladores a lo largo del tiempo.

Actividad 15: Desarrollo en PHP – PDO con POO

Realizar el desarrollo del CRUD para la Tabla Cliente utilizando Programación Orientada a Objetos



Ejercicio 01 de PDO en PHP

PDO

Menú Principal

[Inicio](#) [Cliente](#) [Producto](#) [Venta](#)

Gestor de Clientes

Menú Principal

[Inicio](#) [Cliente](#) [Producto](#) [Venta](#)

[Agregar Cliente](#)

Filtrar por Nombre S.A.C.

N° Orden	Id	Nombre	NumRuc	Dirección	N° Telefono	Opciones
1	7	COMPANY BUSINESSES S.A.C.	20516333694	CALLAO	999888107	Actualizar Eliminar
2	8	COMPANIA INDUSTRIAL DE CARROCERIAS S.A.C.	20538154564	LIMA	999888108	Actualizar Eliminar
3	10	CONSERMET S.A.C.	20481405999	LA LIBERTAD	999888110	Actualizar Eliminar
4	11	CORPORACION CAYMAN S.A.C.	20493190611	LORETO	999888111	Actualizar Eliminar
5	12	CORPORACION METALICA DEL ACERO S.A.C.	20513855941	CALLAO	999888112	Actualizar Eliminar
6	13	CORPORACION WRT S.A.C. - WRT S.A.C.	20536160206	LIMA	999888113	Actualizar Eliminar
7	14	DC EXPRES S.A.C.	20600465377	LA LIBERTAD	999888114	Actualizar Eliminar
8	16	ECOENERGY S.A.C.	20393530244	UCAYALI	999888116	Actualizar Eliminar
9	17	EHF S.A.C.	20600040562	LIMA	999888117	Actualizar Eliminar
10	20	FABRICACION Y SERVICIOS MULTIPLES UCEDA S.A.C.	20481952965	LA LIBERTAD	999888120	Actualizar Eliminar
11	23	FABRICACIONES BRA S.A.C.	20477313214	LA LIBERTAD	999888123	Actualizar Eliminar
12	24	FABRICACIONES METALICAS ARNOLD S.A.C.	20477664513	LA LIBERTAD	999888124	Actualizar Eliminar
13	25	FABRICACIONES METALICAS CARRANZA S.A.C. FAMECA S.A.C.	20132108294	LA LIBERTAD	999888125	Actualizar Eliminar
14	27	FABRICACIONES METALICAS Y SERVICIOS DIESEL S.A.C - FAMEDI S.A.C.	20505769148	LIMA	999888127	Actualizar Eliminar
15	29	FABRIMETAL S.A.C.	20101633820	LIMA	999888129	Actualizar Eliminar
16	33	FACTORIA FACJOCA S.A.C.	20454650043	AREQUIPA	999888133	Actualizar Eliminar
17	42	FAMA ANDINA S.A.C.	20524006945	LA LIBERTAD	999888142	Actualizar Eliminar
18	44	GRUPO C & R VELOZ S.A.C.	20551034021	LIMA	999888144	Actualizar Eliminar
19	45	GRUPO SERMET S.A.C.	20549685316	CALLAO	999888145	Actualizar Eliminar
20	48	IBIMCO PERU S.A.C.	20522599591	LIMA	999888148	Actualizar Eliminar
21	49	INDUSTRIA CARROCERA DEL PERU S.A.C. - INCAPER	20519105676	LIMA	999888149	Actualizar Eliminar
22	50	INDUSTRIA METALICA BULLON S.A.C.	20514745031	LIMA	999888150	Actualizar Eliminar
23	54	INDUSTRIAS ANCALAYO S.A.C.	20568135271	JUNIN	999888154	Actualizar Eliminar

Agregar nuevo Cliente

[Volver](#)

Código del Cliente 115

Nombre MECATRONICA S.A.C.

N° Ruc 20458127400

Dirección LIMA

Telefono 999888101

[Nuevo](#)

Gestión de Producto

Menú Principal

[Inicio](#) [Cliente](#) [Producto](#) [Venta](#)

[Agregar Producto](#)

Descripción

N° de Orden	Código	Descripción	Categoría	Precio S/.	Opciones
1	40	CARNE DE CANGREJO DE BOSTON	PESCADO/MARISCO	18.4	Actualizar Eliminar
2	53	EMPANADA DE CARNE	CARNES	32.8	Actualizar Eliminar

Agregar nuevo Producto

[Volver](#)

Código del Producto

Descripción

Categoría

Precio (S/.)

[Nuevo](#)

Venta

Menú Principal

[Inicio](#) [Cliente](#) [Producto](#) [Venta](#)

Filtrar por Id de Cliente

N° Orden	Id	Cliente	Fecha de Venta
1	8	6	2018-03-27 00:00:00
2	17	6	2018-04-05 00:00:00

Fuentes Bibliograficas

Open Web Application Security Project. (2021). *OWASP Top 10: The ten most critical web application security risks*. <https://owasp.org/www-project-top-ten/>

PHP Group. (s.f.). *PHP manual: PHP Data Objects (PDO)*. PHP.net. <https://www.php.net/manual/en/book.pdo.php>

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Codd, E. F. (1970). *A relational model of data for large shared data banks*. Communications of the ACM, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>

Esta actividad está basada en los métodos de búsqueda de `ProductoDAO.php`, donde se utilizan consultas preparadas con PDO para buscar productos por ID